

Patrones DTO y DAO

José Manuel Sánchez Álvarez

November 13, 2025

Contents

1	Patrón DTO	1
1.1	Generalidades	1
1.2	Ejemplos con Spring Boot	1
1.2.1	Ejemplo 1	1
1.2.2	Ejemplo 2	2
1.2.3	Ejemplo 1 de DTO en Spring Boot con JsonViews	4
1.2.4	Ejemplo 2 de DTO en Spring Boot con JsonViews	5
2	Patrón DAO	7
2.1	Generalidades	7
2.2	Ejemplo con Spring Boot	7
3	Anotaciones Spring Boot	8

:: Minted y enmarcado

1 Patrón DTO

1.1 Generalidades

El patrón DTO (Data Transfer Object) es un patrón de diseño comúnmente utilizado en aplicaciones de Spring Boot que proporciona una forma de transferir datos entre las capas de la aplicación. El DTO se utiliza para transferir datos entre la capa de aplicación y la capa de persistencia (como una base de datos), sin exponer el modelo de datos subyacente o exponer datos sensibles.

1.2 Ejemplos con Spring Boot

Aquí está cómo utilizar el patrón DTO en una aplicación de Spring Boot:

1.2.1 Ejemplo 1

1. Definir una clase DTO: Cree una clase separada que represente los datos que desea transferir entre la capa de aplicación y la capa de persistencia.

```

1 public class UserDTO
2     private
3     private
4     private
5     private
6

```

2. Mapeo entre DTO y Entidad: Cree una clase mapeadora que mappee entre la clase DTO y la clase de entidad (la clase que representa los datos en la capa de persistencia).

```

1 @Mapper
2 public interface UserMapper
3     toDTO
4     toEntity
5

```

3. Utilizar el DTO en la capa de servicio: Utilice el DTO en la capa de servicio para transferir datos entre la capa de aplicación y la capa de persistencia.

```

1 @Service
2 public class UserService
3     private
4     private
5
6     public      createUser
7                 =      toEntity
8                 =      save
9     return      toDTO
10
11

```

En este ejemplo, el UserService utiliza el UserMapper para mapear entre el UserDTO y el User entity, y para transferir datos entre la capa de aplicación y la capa de persistencia. De esta manera, los datos sensibles en el User entity no se exponen a la capa de aplicación, y la transferencia de datos es más segura.

1.2.2 Ejemplo 2

1. Clase usuario

```

1 @Entity
2 public class User
3     @Id
4     @GeneratedValue = IDENTITY
5     private
6     private
7     private
8     private
9     private
10

```

2. Repositorio:

```

1 @Repository
2 public interface UserRepository extends < >

```

3
4

3. Mapeador DTO:

```
1 @Component
2 public class UserMapper
3     public      toDTO
4         = new
5         setUsername      getUsername
6         setEmail      getEmail
7         setPassword      getPassword
8         setPhoneNumber      getPhoneNumber
9     return
10
11
12     public      toEntity
13         = new
14         setUsername      getUsername
15         setEmail      getEmail
16         setPassword      getPassword
17         setPhoneNumber      getPhoneNumber
18     return
19
20
```

4. Servicio (lo normal es usar un servicio y no hacer las cosas directamente en el controlador, cuando es mucho trabajo)

```
1 @Service
2 public class UserService
3     private
4     private
5
6     public UserService
7
8         this userRepository =
9         this userMapper =
10
11
12     public      createUser
13         =      toEntity
14         =      save
15     return      toDTO
16
17
18     public      <      > getAllUsers
19         <      >      =      findAll
20     return      stream
21
22         toList
23
24
25
```

5. Controlador Rest:

```

1  @RestController
2  @RequestMapping("/api/users")
3  public class UserController
4      private
5
6      public UserController
7          this.userService =
8
9
10     @PostMapping
11     public      createUser @RequestBody
12         return      createUser
13
14
15     @GetMapping
16     public      <      > getAllUsers
17         return      getAllUsers
18
19

```

En este ejemplo, se utilizan las anotaciones `@Entity` para definir una entidad `User`, `@Repository` para crear un repositorio `UserRepository` que extienda de `JpaRepository`, `@Component` para crear un mapeador de DTO `UserMapper`, `@Service` para crear un servicio `UserService` que utiliza el repositorio y el mapeador, y finalmente `@RestController` para crear un controlador REST `UserController` que utiliza el servicio para manejar las solicitudes de API relacionadas con los usuarios.

Este ejemplo muestra cómo se puede utilizar Spring Boot para implementar el patrón DAO y separar la lógica de negocios y acceso a datos en capas separadas. Al utilizar DTOs y mapeadores, se logra una mayor flexibilidad y seguridad en la representación de los datos en la API.

- Modelo de entidad: En lugar de utilizar DTOs, se pueden exponer directamente las entidades JPA en la API.
- Modelo de transferencia de objetos dinámicos: En lugar de utilizar DTOs estáticos, se pueden utilizar objetos dinámicos como mapas o JSON para representar los datos.
- Modelo de transferencia de objetos simples: En lugar de utilizar DTOs complejos, se pueden utilizar objetos simples con propiedades básicas para representar los datos.
- Modelo de transferencia de objetos anotados: En lugar de utilizar DTOs, se pueden utilizar objetos anotados con Jackson o similar para controlar la representación de los datos en la API.

1.2.3 Ejemplo 1 de DTO en Spring Boot con JsonViews

Aquí hay un ejemplo de código que utiliza `JsonView` y objetos anotados para crear un DTO en Spring Boot:

```

1  public class UserDto
2      public interface PublicView
3      public interface InternalView extends
4

```

```

5      @JsonView          class
6      private
7
8      @JsonView          class
9      private
10
11     @JsonView          class
12     private
13
14     // getters and setters for id, username, and password
15
16
17     @RestController
18     public class UserController
19         @GetMapping "/user/{id}"
20         @JsonView      PublicView class
21         public         getUser @PathVariable
22                         = // retrieve user data
23
24         return
25
26         @PostMapping "/user"
27         @JsonView      InternalView class
28         public         createUser @RequestBody
29                         // create user
30         return
31
32

```

En este ejemplo, se utiliza JsonView para especificar las vistas publica y interna para el DTO de usuario. En el controlador, se usan las anotaciones @JsonView en los métodos GET y POST para controlar qué información se incluye en la respuesta JSON. De esta manera, se puede garantizar que la información confidencial, como la contraseña, solo sea visible en la vista interna.

1.2.4 Ejemplo 2 de DTO en Spring Boot con JsonViews

ejemplo completo de código que utiliza JsonView, objetos anotados y un repositorio para crear un DTO en Spring Boot:

```

1      public class UserDto
2          public interface PublicView
3          public interface InternalView extends
4
5          @JsonView          class
6          private
7
8          @JsonView          class
9          private
10
11         @JsonView          class
12         private
13
14         // getters and setters for id, username, and password
15

```

```

16
17 @Entity
18 public class User
19     @Id
20     @GeneratedValue          =          IDENTITY
21     private
22
23     private
24     private
25
26     // getters and setters for id, username, and password
27
28
29 @Repository
30 public interface UserRepository extends          <          >
31
32 @Component
33 public class UserMapper
34     public          toDto
35         = new
36         setId          getId
37         setUsername          getUsername
38         setPassword          getPassword
39     return
40
41
42     public          toEntity
43         = new
44         setId          getId
45         setUsername          getUsername
46         setPassword          getPassword
47     return
48
49
50
51 @RestController
52 public class UserController
53     private final
54     private final
55
56     public UserController
57
58
59
60     this userRepository =
61     this userMapper =
62
63
64     @GetMapping "/user/{id}"
65     @JsonView          PublicView class
66     public          getUser @PathVariable
67         =
68
69     findById

```

```

70         -> new
71         return toDto
72
73
74     @PostMapping("/user")
75     @JsonView({InternalView.class})
76     public ResponseEntity<User> createUser(@RequestBody User user) {
77         User entity = user.toEntity();
78         entity = repository.save(entity);
79         return ResponseEntity.ok(entity.toDto());
80     }
81

```

En este ejemplo, se utiliza un repositorio JPA para acceder a los datos de usuario y un mapeador para convertir entre el DTO y el modelo de entidad. El controlador utiliza las anotaciones @JsonView y la lógica de repositorio y mapeo para recuperar y crear usuarios en la API.

2 Patrón DAO

2.1 Generalidades

El patrón DAO (Data Access Object) es un patrón de diseño que se utiliza para separar la lógica de acceso a datos de la lógica de negocios de una aplicación. Con el DAO, se puede proporcionar una interfaz común para acceder a diferentes fuentes de datos, como bases de datos relacionales, archivos, APIs de terceros, etc. El DAO permite a la aplicación interactuar con la fuente de datos sin tener que conocer los detalles de la implementación de la fuente de datos. Esto permite a la aplicación ser más flexible y fácil de mantener, ya que se pueden realizar cambios en la fuente de datos sin afectar la aplicación.

2.2 Ejemplo con Spring Boot

1. Crear una interfaz DAO: Esta interfaz define los métodos para acceder a la fuente de datos, como insertar, actualizar, eliminar y recuperar registros.

```

1 public interface UserDao {
2     void insert(User user);
3     User getById(int id);
4     void update(User user);
5     void delete(int id);
6 }

```

1. Implementar la interfaz DAO: Esta clase proporciona una implementación concreta de los métodos definidos en la interfaz DAO.

```

1 @Repository
2 public class UserDaoImpl implements UserDao {
3     @Autowired
4     private UserRepository repository;
5     // implementation of CRUD methods
6 }

```

1. Inyectar DAO en la clase de servicio: La clase de servicio usa la implementación DAO para acceder a la fuente de datos.

```

1  @Service
2  public class UserService
3  @Autowired
4  private
5  // business logic using UserDao
6

```

En este ejemplo, se utiliza JDBC como fuente de datos y JdbcTemplate como herramienta de acceso a datos en Spring Boot. Además, se usan anotaciones como @Repository y @Service para identificar las clases como componentes de Spring.

- Ventajas: Mejora la separación de responsabilidades, facilita la mantenibilidad y pruebas unitarias de la aplicación.
- Inconvenientes: Aumenta la complejidad de la aplicación y puede ser difícil de implementar correctamente.

Alternativas:

- JPA (Java Persistence API): una especificación Java para acceder a bases de datos relacionales.
- Hibernate: una popular implementación de JPA.
- JDBC (Java Database Connectivity): una API de Java para acceder a bases de datos relacionales.

3 Anotaciones Spring Boot

Las siguientes son las anotaciones comúnmente utilizadas en Spring Boot:

- @SpringBootApplication: es una combinación de varias anotaciones, incluidas @Configuration, @EnableAutoConfiguration, y @ComponentScan. Se usa para habilitar la configuración automática en una aplicación y para permitir a Spring escanear los componentes en el paquete raíz y sus subpaquetes.
- @RestController: se utiliza para crear un controlador REST en una aplicación Spring Boot.
- @Repository: se utiliza para identificar una clase como un repositorio, es decir, una clase que proporciona acceso a datos.
- @Service: se utiliza para identificar una clase como un servicio, es decir, una clase que contiene la lógica de negocios de una aplicación.
- @Autowired: se utiliza para inyectar un objeto de una dependencia en una clase.
- @Entity: se utiliza en combinación con JPA para identificar una clase como una entidad, es decir, una clase que representa una tabla en una base de datos relacional.
- @Transactional: se utiliza para controlar las transacciones en una aplicación. Esta anotación garantiza que las operaciones en una transacción se realicen juntas o no se realicen en absoluto.

Estas son solo algunas de las anotaciones más comunes utilizadas en Spring Boot. Hay muchas más anotaciones disponibles, dependiendo de los requisitos específicos de la aplicación.